

KLH Bachupally

AI-Based Phishing Detection System Using Ensemble Learning

Academic Project and Experiment Report

Department	DEPARTMENT OF CSIT
Academic Year	2026 - 2027
Project Guide	Dr. K Venkateshwara Rao

Project Team

2320090017	D Sunidhi
2320090060	P Manvitha
2320090069	G Likith

Character-Level CNN and LightGBM Ensemble for Safe URL-String Analysis

Contents

Contents	2
1. Abstract	3
2. Introduction and Problem Statement	3
3. Objectives	3
4. Related Work and Reference Study	4
5. Dataset, Preprocessing, and Leakage Prevention	5
6. Feature Engineering	6
7. Model Architectures and Training	7
8. Ensemble Methodology and Experimental Protocol	8
9. Final Held-Out Test Results	9
10. Visual Experimental Evidence	10
11. Threshold-Independent Evaluation	11
12. Feature and Probability Analysis	12
13. Working Local Application	13
14. Synthetic Suspicious-String Demonstration	14
15. Reference Comparison and Implementation Evidence	15
16. Limitations, Future Work, and Conclusion	16
17. References	17

Report scope

This document reports an independent limited-compute reproduction using 20,000 URLs. Reference-study measurements are labelled separately and are never presented as outcomes of the local experiment.

1. Abstract

Phishing URLs imitate legitimate services to steal credentials or other sensitive information. Blacklists and fixed rules remain useful for known threats, but newly created links may evade them. This project implements a URL-only phishing detector that combines a Character-Level Convolutional Neural Network (Char-CNN), which learns sequential patterns directly from character strings, with Light Gradient Boosting Machine (LightGBM), which uses 36 engineered lexical, structural, and statistical URL features. The work is an independent limited-compute reproduction inspired by the PhishX reference study. A balanced deterministic subset of 20,000 URLs - 10,000 legitimate and 10,000 phishing - was constructed from frozen public snapshots. Root-domain-separated train, validation, and test partitions prevented domain leakage, and all model decisions were frozen before the held-out test set was evaluated. The validation-selected 95/5 CNN-LightGBM ensemble achieved 99.525% accuracy, 100% precision, 99.048% recall, and 99.931% ROC-AUC on 3,998 held-out URLs. A local FastAPI application applies the same saved preprocessing and models in real time while treating every submitted URL only as text; it never visits the destination. The completed artifacts, saved models, predictions, tests, and audit records demonstrate that the hybrid method can be reproduced on modest local hardware without overstating its scope or equivalence to the full reference study.

2. Introduction and Problem Statement

A phishing URL can use misleading domains, subdomains, paths, punctuation, encoding, or brand-like tokens to appear trustworthy. Reputation lists may not yet contain a new attack, while fixed rules can be evaded through small lexical changes. Machine learning offers a complementary first-stage defense by learning patterns that generalize beyond exact blacklist matches. URL-only analysis is particularly useful because it can produce a rapid risk estimate without downloading potentially hostile content.

Feature-based models explicitly represent known warning signals but may miss complex character sequences. Character-based deep learning can learn such sequences directly but does not make engineered indicators explicit. This project investigates whether combining both probability estimates provides a robust, reproducible detector under limited local compute.

3. Objectives

- Create a balanced, traceable 20,000-URL dataset and analyse URLs safely as text.
- Implement 36 engineered features and released-compatible 200-character tokenization.
- Train and validate LightGBM and Char-CNN models independently.
- Prevent root-domain leakage and preserve a genuinely held-out test set.
- Select ensemble weights using validation ROC-AUC before test evaluation.
- Build a local web interface using the saved preprocessing and models.

4. Related Work and Reference Study

The principal reference is the PhishX study by Dubey, Tripathi, Srivastava, and Singh. Its hybrid design combines a character-level neural network with engineered URL features and LightGBM. URLNet by Le et al. motivates direct learned representations of malicious URLs, while work by Garera et al., Blum et al., and Yang et al. illustrates the value of lexical, structural, and multidimensional phishing indicators.

Reference architecture

Raw URL
Character branch: URL -> Char-CNN -> probability
Feature branch: URL -> engineered features -> LightGBM -> probability
Weighted probability combination -> prediction

Reference-study results

Dataset	Accuracy	Precision	Recall	ROC-AUC
Approximately 99,361 URLs	99.819%	100%	99.635%	99.947%

Research-integrity boundary

The values above are REFERENCE STUDY RESULTS. The local reproduction used a separate deterministic 20,000-URL subset, stricter domain-separated splits, its own model runs, and independently stored metrics.

5. Dataset, Preprocessing, and Leakage Prevention

Frozen CSV snapshots were obtained from the released PhishX repository. The phishing snapshot was based on PhishTank and contained 49,363 unique cleaned URLs after eight duplicates were removed. The legitimate snapshot was based on Tranco and contained 50,000 unique URLs. These available snapshot totals are distinct from the reference paper's approximately reported 99,361-URL corpus.

Quantity	Legitimate	Phishing	Total
Available unique frozen URLs	50,000	49,363	99,363
Used in this experiment	10,000	10,000	20,000

Preprocessing

Nulls and duplicates were audited, each class was sampled deterministically with seed 42, and labels were defined as 0 for legitimate and 1 for phishing. Character input was lowercased and encoded to length 200 using PAD 0, 49 explicit characters at indices 1-49, and UNK 50. Long URLs retain the rightmost 200 characters; shorter strings are right-padded with zeros.

Domain-leakage correction

The first stratified split placed URLs sharing a registered root domain in different partitions. That could allow domain-specific patterns learned during training to reappear in testing and inflate performance. The problem was detected and corrected before either model was trained.

Split pair	Initial overlapping domains	Final overlapping domains
Train - validation	153	0
Train - test	232	0
Validation - test	108	0

Final partition	Rows	Legitimate	Phishing
Train	14,002	6,997	7,005
Validation	2,000	1,000	1,000
Held-out test	3,998	2,003	1,995

Safety design

Dataset and submitted URLs were never visited. Feature extraction performed no DNS lookup, external HTTP request, webpage download, or JavaScript execution; each URL was treated only as text.

6. Feature Engineering

The LightGBM branch receives exactly 36 numeric features implemented in `src/features.py`. The design combines compact interpretable indicators with the CNN's automatically learned character patterns.

Category	Implemented examples	Purpose
Lexical	URL, host and path length; digit and letter counts; digit-letter ratio; special characters; hyphens	Measure character composition and obfuscation.
Structural	Dots, path segments, query parameters, HTTPS, port, fragment, at-sign, IP host	Describe URL hierarchy and unusual structure.
Statistical/domain	Hostname entropy, path entropy, vowel fraction, suspicious TLD	Represent randomness, linguistic form, and domain indicators.
Length buckets	0-20, 21-40, 41-60, 61-80, 81-100, 101+	Expose broad URL-length ranges.
Suspicious tokens	login, signin, secure, webscr, bank, verify, update, account, confirm	Mark common credential and account lures.

Interpretation boundaries

No single feature is treated as proof of phishing. HTTPS, for example, is not automatically legitimate, and a long URL is not automatically malicious. LightGBM combines thresholds and interactions across the feature set. Feature importance describes tree usage rather than causal security evidence.

Character representation

Property	Value
Explicit alphabet	49 letters, digits, and URL symbols
Effective indices	PAD 0; explicit 1-49; UNK 50
Sequence length	200
Truncation	Keep rightmost 200 characters
Padding	Zeros on the right

7. Model Architectures and Training

Character-Level CNN

Stage	Layer	Configuration
1	URL character encoding	200 indices
2	Embedding	16 dimensions
3	Parallel Conv1D	kernels 3, 5, 7; 128 filters each
4	Activation and pooling	ReLU; adaptive max pooling
5	Concatenation	384 dimensions
6	Dense and dropout	384 -> 64; ReLU; dropout 0.3
7	Output	64 -> 1 binary logit

The network has 56,625 trainable parameters. It used Adam with learning rate 0.001, batch size 64, and Apple MPS acceleration. Early stopping patience was two epochs; seven epochs completed, the best checkpoint was epoch 5, and training took approximately 16.9 seconds.

LightGBM

LightGBM builds a sequence of decision trees over the 36 engineered features. It efficiently models nonlinear thresholds and feature interactions on CPU. Sigmoid calibration was applied after fitting to improve probability interpretation; calibration was not expected to maximize fixed-threshold accuracy.

Objective	Max estimator	Learning rate	Leaves	Best iteration	Training time
Binary	1,000	0.05	64	88	0.84 s

Validation control

LightGBM early stopping used validation ROC-AUC with patience 50. CNN checkpoint selection used validation performance with patience 2. Neither model used held-out test outcomes for fitting or model selection.

8. Ensemble Methodology and Experimental Protocol

Probability combination

Ensemble equation

$$P(\text{ensemble}) = w(\text{CNN}) \times P(\text{CNN}) + w(\text{LightGBM}) \times P(\text{LightGBM})$$

The reference weighting was 0.60 CNN and 0.40 LightGBM. The local validation search evaluated CNN weights from 0.00 to 1.00 in steps of 0.05 and used validation ROC-AUC as the selection metric. The selected configuration was 0.95 CNN and 0.05 LightGBM, with validation ROC-AUC 99.7066%. The gain was small and is not presented as a major improvement.

Configuration	CNN weight	LightGBM weight	Selection basis
Reference ensemble	0.60	0.40	Reference methodology
Selected ensemble	0.95	0.05	Highest validation ROC-AUC

Train, validation, and test roles

Partition	Permitted role
Train	Fit LightGBM trees and Char-CNN parameters.
Validation	Early stopping, LightGBM probability calibration, and ensemble weight selection.
Held-out test	One final evaluation after preprocessing, models, calibration, weights, and threshold were frozen.

Held-out evaluation rule

The classification threshold was fixed at 0.50. No threshold, weight, architecture, or training decision was changed after test results were observed. Selecting a better-looking test configuration afterward would constitute test-set tuning.

9. Final Held-Out Test Results

Model	Accuracy	Precision	Recall	F1	ROC-AUC	PR-AUC	FP	FN
Calibrated LightGBM	99.600%	99.749%	99.449%	99.598%	99.851%	99.908%	5	11
Char-CNN	99.525%	100.000%	99.048%	99.522%	99.945%	99.955%	0	19
Reference ensemble 60/40	99.575%	99.949%	99.198%	99.572%	99.892%	99.928%	1	16
Selected ensemble 95/5	99.525%	100.000%	99.048%	99.522%	99.931%	99.948%	0	19

Selected ensemble confusion matrix

True negatives	False positives	False negatives	True positives
2003	0	19	1976

Interpretation

Calibrated LightGBM produced the highest test accuracy (99.600%) and recall (99.449%). Char-CNN produced the highest test ROC-AUC (99.945%). The validation-selected ensemble achieved zero false positives and ROC-AUC 99.931%, but its fixed-threshold decisions matched the CNN on this test set. The 95/5 ensemble is retained because it was selected before test inspection, not because it dominated every final metric.

Zero false positives means no legitimate held-out URL was incorrectly blocked. Nineteen phishing URLs were missed, illustrating the security tradeoff between user disruption and attack recall. The threshold was not changed after observing these errors.

10. Visual Experimental Evidence

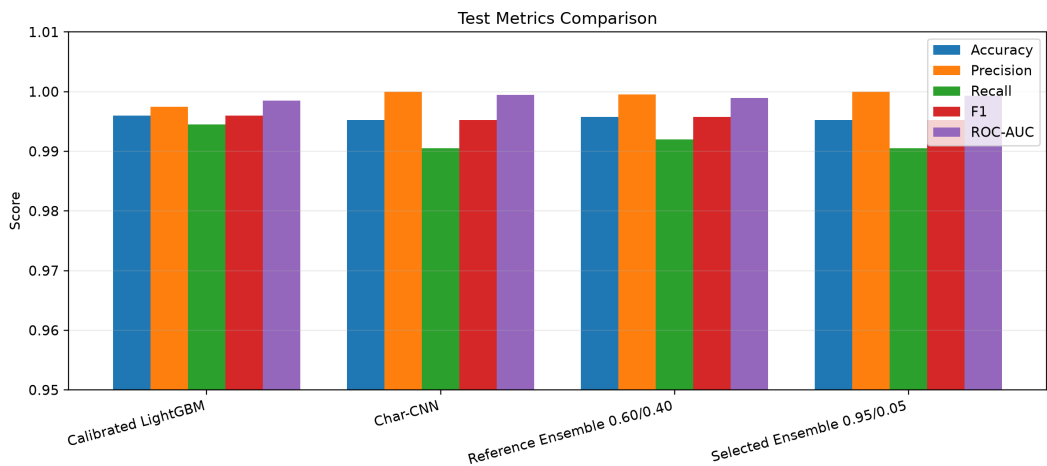


Figure 1. Comparison of component and ensemble performance on the final held-out test set.

Differences are small but metric-dependent: LightGBM leads test accuracy, while Char-CNN leads ROC-AUC. The chart supports multi-metric reporting rather than a single winner claim.

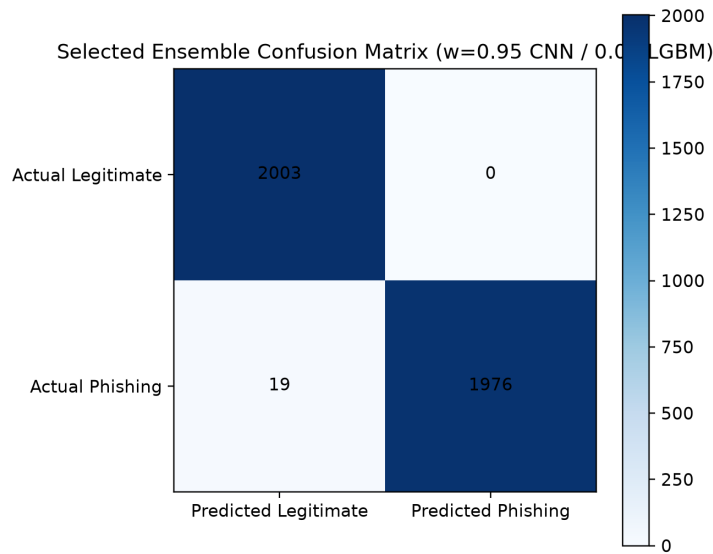


Figure 2. Confusion matrix of the validation-selected 95/5 ensemble on 3,998 held-out URLs.

At threshold 0.50, the ensemble produced 2,003 true negatives, zero false positives, nineteen false negatives, and 1,976 true positives.

11. Threshold-Independent Evaluation

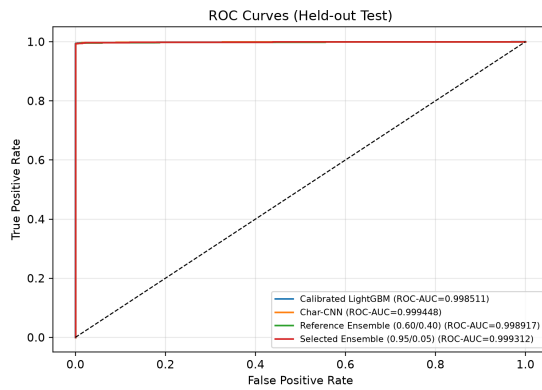


Figure 3. Receiver operating characteristic curves on held-out data.

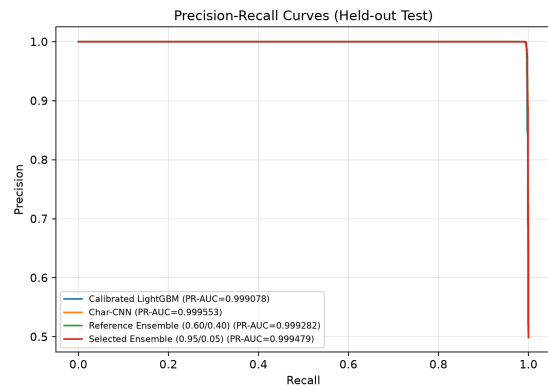


Figure 4. Precision-recall curves on held-out data.

Interpretation

The ROC curves summarize false-positive and true-positive tradeoffs across thresholds, while precision-recall curves focus on phishing detection quality as the decision threshold changes. All configurations rank the two classes strongly, with final ROC-AUC values between 99.851% and 99.945%.

Threshold-independent curves do not replace the fixed-threshold confusion matrix. A deployment must still decide the relative cost of false alerts and missed attacks using validation or operational data, not the held-out test set.

Metric context

ROC-AUC evaluates ranking over thresholds. PR-AUC emphasizes precision and recall. Accuracy and confusion counts describe the chosen 0.50 operating point. Reporting all of them prevents a narrow interpretation of performance.

12. Feature and Probability Analysis

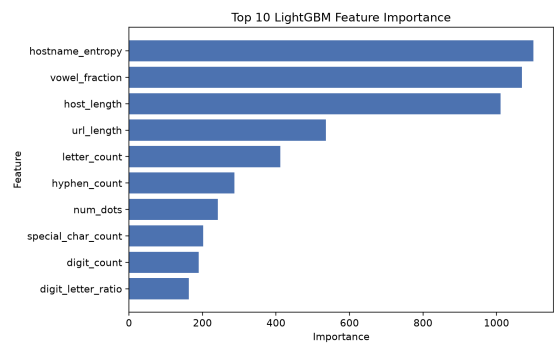


Figure 5. Leading engineered features by LightGBM importance.

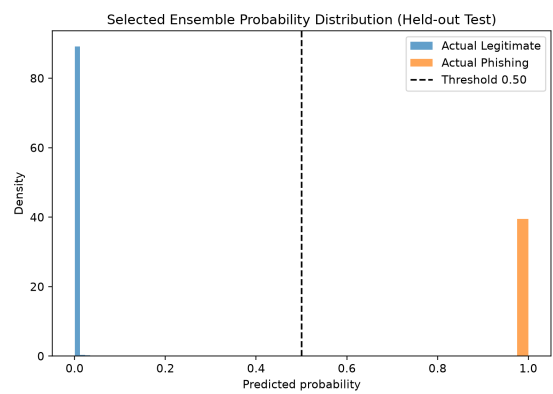


Figure 6. Held-out phishing-probability distributions.

Leading observed features

Rank	Feature	Importance	Rank	Feature	Importance
1	hostname_entropy	1100	6	hyphen_count	287
2	vowel_fraction	1069	7	num_dots	242
3	host_length	1011	8	special_char_count	202
4	url_length	536	9	digit_count	190
5	letter_count	412	10	digit_letter_ratio	163

Hostname entropy, vowel fraction, and host length were the strongest recorded LightGBM contributors, followed by URL length and character-count features. These values indicate tree-split usage, not causality. The probability distribution shows strong overall class separation while retaining an ambiguous region where threshold-dependent errors occur.

13. Working Local Application

The demonstration application uses FastAPI, HTML, CSS, and vanilla JavaScript. GET /health reports readiness, and POST /predict accepts a URL string. The server applies the same 36-feature extractor, 200-character tokenizer, calibrated LightGBM model, Char-CNN checkpoint, and frozen ensemble weights used in evaluation. It is a local application at `http://127.0.0.1:8000`, not a public deployment.

College Reproducibility Demo

AI-Based Phishing Detection System

Character-Level CNN + LightGBM Ensemble

Enter a URL to analyse

ANALYSE URL

Prediction

LEGITIMATE
Confidence: 99.4%
Selected Ensemble: 0.6% phishing probability
Input URL: https://www.google.com

Model Probability Breakdown

- Char-CNN: 0.5%
- LightGBM: 1.7%
- Reference Ensemble (60/40): 1.0%
- Selected Ensemble (95/5): 0.6%

URL Signals

- URL Length: 22
- Host Length: 14
- Hostname Entropy: 2.8424
- Number of Dots: 2
- Digit/Letter Ratio: 0
- Number of Dots in Path: 0
- Suspicious TLD: No
- Contains Login Keyword: No
- Contains Verify Keyword: No

► About the Model

Figure 7. Local application correctly classifying `https://www.google.com` as legitimate and displaying component probabilities and URL signals.

The selected ensemble reports approximately 0.6% phishing probability for this example. One example is illustrative only and is not used as a separate performance benchmark.

14. Synthetic Suspicious-String Demonstration

College Reproducibility Demo

AI-Based Phishing Detection System

Character-Level CNN + LightGBM Ensemble

Enter a URL to analyse

ANALYSE URL

Prediction

PHISHING
Confidence: 100.0%
Selected Ensemble: 100.0% phishing probability
Input URL: http://secure-account-login-example.xyz/verify

Model Probability Breakdown

- Char-CNN: 100.0%
- LightGBM: 99.9%
- Reference Ensemble (60/40): 100.0%
- Selected Ensemble (95/5): 100.0%

URL Signals

- URL Length: 46
- Host Length: 32
- Hostname Entropy: 4.0778
- Number of Dots: 1
- Digit/Letter Ratio: 0
- Number of Dots in Path: 1
- Suspicious TLD: Yes
- Contains Login Keyword: Yes
- Contains Verify Keyword: Yes

About the Model

Figure 8. Local application classifying the synthetic suspicious-looking string `http://secure-account-login-example.xyz/verify` as phishing. The URL was analysed only as text and was not visited.

Application safety

Inference performs no DNS lookup, external HTTP request, webpage download, script execution, or browser navigation to the submitted address. The screenshot records a local string-classification result, not a visit to a phishing website.

The component models and both ensemble configurations assigned approximately 99.9-100.0% phishing probability to this synthetic string. This is a useful demonstration of the interface and signal presentation, but it must not be generalized beyond the held-out quantitative evaluation.

15. Reference Comparison and Implementation Evidence

Measure	Reference study	Local reproduction
Dataset	Approximately 99,361 URLs	20,000 URLs
Accuracy	99.819%	99.525%
Precision	100%	100%
Recall	99.635%	99.048%
ROC-AUC	99.947%	99.931%

The local selected-ensemble accuracy is approximately 0.294 percentage points lower. Plausible factors include the smaller deterministic subset, strict domain separation, different sampling, local implementation details, and limited training and tuning. The results are close, but the studies are not directly equivalent and the local system is not claimed to be better.

Implementation Evidence

Evidence	Repository artifact
Dataset provenance	DATA_PROVENANCE.md
Experiment history	RUN_LOG.md
Feature engineering and tokenizer	src/features.py; src/char_tokenizer.py
LightGBM and CNN implementation	src/train_lightgbm.py; src/train_charcnn.py
Ensemble selection	src/select_ensemble.py; results/ensemble_config.json
Final held-out metrics	results/final_test_metrics.json
Saved models	models/lightgbm_calibrated.pkl; models/char_cnn.pt
Application inference	app/inference.py
Automated tests	24 passed; 0 failed

Auditability

The repository retains source implementations, saved indices, trained artifacts, validation history, ensemble search results, final predictions, plots, tests, and chronological run notes. Reproducibility is visible without embedding raw logs or source-code pages in this report.

16. Limitations, Future Work, and Conclusion

Limitations

- The experiment used a balanced 20,000-URL subset rather than the complete available snapshots.
- Detection is URL-only and does not inspect webpage content, certificates, redirects, or visual similarity.
- No live reputation, WHOIS, domain-age, or hosting-history signals were used.
- The historical snapshots do not establish temporal generalization to future campaigns.
- Adversarial robustness was not deeply tested, and balanced-test performance may differ under operational class imbalance.
- The selected ensemble did not dominate every component metric.

Future work

Future evaluation could use larger and newer frozen datasets, time-based splits, adversarial URL transformations, validation-driven security thresholds, and safe optional domain-reputation or webpage-content services. SHAP explanations could improve LightGBM interpretability, and a browser extension could provide a practical client. None of these extensions is claimed as completed work.

Conclusion

The project reproduced the core hybrid CNN and LightGBM methodology on limited local hardware using 20,000 URLs. It implemented 36 engineered features, exact character metadata, zero-overlap domain-separated evaluation, independently trained components, validation-only ensemble selection, and a working local application. The final selected ensemble achieved 99.525% accuracy, 100% precision, 99.048% recall, and 99.931% ROC-AUC on 3,998 held-out URLs. These results demonstrate a strong reproducible academic prototype without claiming equivalence to or superiority over the reference study.

Reproducibility

Run the completed project from its repository root:

```
source .venv/bin/activate
pytest
uvicorn app.main:app --reload
```

Local application: <http://127.0.0.1:8000>

17. References

1. R. Dubey, A. M. Tripathi, A. Srivastava, and S. Singh, "Phishing Detection System: An Ensemble Approach Using Character-Level CNN and Feature Engineering," arXiv:2512.16717, 2025. Reference implementation: <https://github.com/dubeyrudra-1808/PhishX>.
2. PhishTank, PhishTank Valid Phishing URLs. <https://phishtank.org/>.
3. Tranco, Tranco Top 1M List. <https://tranco-list.eu/>.
4. H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, "URLNet: Learning a URL representation with deep learning for malicious URL detection," arXiv:1802.03162, 2018.
5. S. Garera, N. Provos, M. Chew, and A. D. Rubin, "A framework for detection and measurement of phishing attacks," Proceedings of the 2007 ACM Workshop on Large Scale Attack Defence, pp. 1-8, 2007.
6. A. Blum, B. Wardman, T. Solorio, and G. Warner, "Lexical feature-based phishing URL detection using online learning," Proceedings of the 3rd ACM Workshop on Artificial Intelligence and Security, pp. 54-60, 2010.
7. P. Yang, G. Zhao, and P. Zeng, "Phishing website detection based on multidimensional features driven by deep learning," IEEE Access, vol. 7, pp. 15196-15209, 2019.

Reproduction statement

This report documents an independent educational reproduction. Reference metrics, local metrics, public data provenance, and implementation evidence are presented as separate, auditable quantities.